FIREBRAND
LYCEUM EDUCATION GROUP

Module 4: Software Design
Session 1 stretch: thinkers behind design choices

Software Developer Level 4



Parnas: hide what is likely to change

The idea (Parnas, 1972)

Split a system so that each module hides one design decision that is likely to change, such as a data format or a supplier's interface.

Other modules use it only through its interface, so that decision can change in one place.

Why it matters to a developer

It is the reasoning behind component-based design.

Splitting by processing steps feels natural, but ties every module to the same decisions, so one change ripples everywhere.

Try this: which portal decision is most likely to change? Which module should hide it?

FIREBRAND
LYCEUM EDUCATION GROUP

Conway's law: systems mirror their teams

The idea (Conway, 1968)

A team that designs a system tends to produce a design that copies its own communication structure.

Why it matters

Interfaces appear where teams rarely talk; parts blur where they talk every day.

Try this

Kestrel Moor has four in-house developers and an external agency. Where will the portal's seams fall?

Contested: an observation, not a law of nature. Evidence for it is mixed.



FIREBRAND
LYCEUM EDUCATION GROUP

Norman: show what is possible, then confirm it

Why it matters: if tenants cannot see what to do, or cannot tell it worked, they phone instead.

AFFORDANCES AND SIGNIFIERS

Make the possible action obvious

An affordance is what something lets you do; a signifier tells you it is there.

A photo button that looks like a button gets pressed.

FEEDBACK

Tell people what just happened

Every action needs a visible, immediate result.

Try this: what feedback does a tenant get today after sending the web form? (Case Brief Part 1)

FIREBRAND
LYCEUM EDUCATION GROUP

Evans: one language for experts and code

A GLOSSARY ENTRY: RIVERSIDE LEISURE CENTRE

(1) Term: class session

(2) Means: one run of a class at a set time in a set studio (Spin, Tuesday 18:00, Studio 2).

(3) Not the same as: class type, the kind of class (Spin).

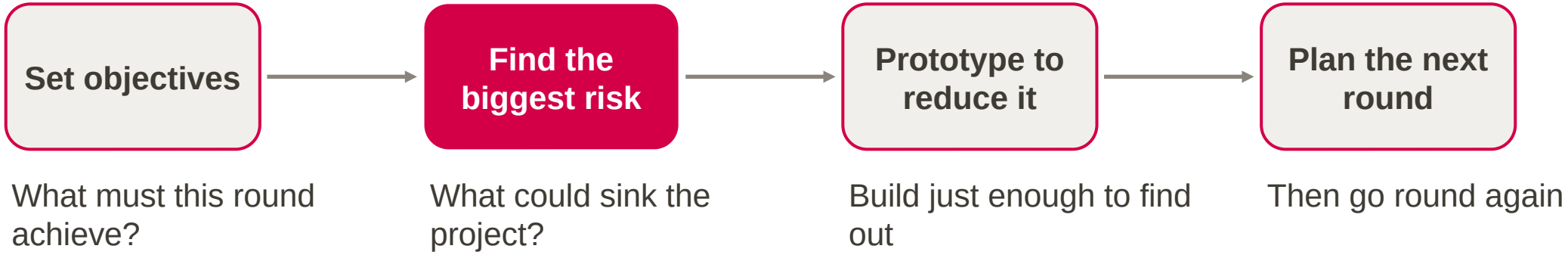
(4) Agreed with: the bookings manager and the instructors.

1. The idea: experts and developers use the same words in talk, diagrams and code (Evans, 2003).
2. Define each term once, in the experts' words.
3. Why it matters: one word with two meanings becomes two bugs.
4. Try this: write the entry for 'Emergency'. Who must agree it?

FIREBRAND
LYCEUM EDUCATION GROUP

Boehm's spiral: let risk choose the next step

Why it matters: design effort goes where failure would cost most (Boehm, 1988).



Try this: what is the biggest risk in the portal's first slice, and what is the cheapest prototype that would reduce it?

FIREBRAND LYCEUM EDUCATION GROUP

Brooks: no silver bullet

ESSENCE AND ACCIDENTS

> **The hard part of software is deciding what to build, not writing it in code.**

Brooks (1987) separates essential complexity, in the problem itself, from accidental complexity, from our tools. Better tools mostly remove the accidental kind.

Try this: which part of the portal stays hard whatever language or AI tool you use? Why?

Contested: some argue newer tools have shifted the balance.

FIREBRAND
LYCEUM EDUCATION GROUP

FIREBRAND
LYCEUM EDUCATION GROUP

Module 4: Software Design
Session 1 stretch: thinkers behind design choices



Software Developer Level 4

Optional self-study for after Session 1, about 30 minutes. Nothing here is needed for the live session or the Consolidate Quiz.

Six ideas behind today's session, one per slide: Parnas on information hiding, Conway's law, Norman on signifiers and feedback, Evans on a shared domain language, Boehm's risk-driven spiral and Brooks's 'No Silver Bullet'. Each slide gives the idea in plain words and why it matters to a developer, then a 'Try this' question to apply it to Kestrel Moor Homes or to your own project. These notes repeat the idea, say why it matters, add a hint on what a good answer to 'Try this' does, and give the full reference.

Two of the ideas are contested, and the slides say so: Conway's law is an observation with mixed evidence, and how far Brooks's argument still holds with today's tools is debated.

If you use one of these ideas in your own work or in a conversation with your skills coach, cite the original source in the notes, not this deck. Keep any workplace examples general: no personal or customer data, credentials or confidential material.

Parnas: hide what is likely to change

The idea (Parnas, 1972)

Split a system so that each module hides one design decision that is likely to change, such as a data format or a supplier's interface.

Other modules use it only through its interface, so that decision can change in one place.

Why it matters to a developer

It is the reasoning behind component-based design.

Splitting by processing steps feels natural, but ties every module to the same decisions, so one change ripples everywhere.

Try this: which portal decision is most likely to change? Which module should hide it?

FIREBRAND LYCEUM EDUCATION GROUP

2

The idea. Parnas compared two ways of splitting a small program into modules: by processing steps (input, sort, output) and by design decisions that are likely to change, such as how data is stored. The second way made change cheaper, because each decision lived in one module and other modules used it only through an interface. This is the origin of 'information hiding'.

Why it matters. It is the reasoning behind component-based design. When you split a system, ask what is likely to change, not what happens first, second and third.

Try this. Read Case Brief Parts 1, 2 and 5 and list the decisions that could change during the portal's life. Pick the one most likely to change. A good answer names the decision, the module that would hide it and what the rest of the system would see through its interface. Naming a technology, such as 'use microservices', is not an answer on its own: say which decision it hides.

Reference: Parnas, D. L. (1972) 'On the criteria to be used in decomposing systems into modules', Communications of the ACM, 15(12), pp. 1053 to 1058.

Conway's law: systems mirror their teams

The idea (Conway, 1968)
A team that designs a system tends to produce a design that copies its own communication structure.

Why it matters
Interfaces appear where teams rarely talk; parts blur where they talk every day.

Try this
Kestrel Moor has four in-house developers and an external agency. Where will the portal's seams fall?

Contested: an observation, not a law of nature. Evidence for it is mixed.



FIREBRAND
LYCEUM EDUCATION GROUP

3

The idea. Conway argued that any organisation that designs a system will produce a design whose structure copies the organisation's communication structure. Fred Brooks later called it Conway's law.

Why it matters. Interfaces tend to appear where teams rarely talk, and parts blur where people talk every day. If you know how the teams are organised, you can predict where the seams in a design will fall, and ask whether they are in the right place.

Contested. It is an observation, not a proven law. Some studies find a link between how teams communicate and how software is structured; others find it weak or indirect. Use it as a prompt for questions, not as a rule.

Try this. Case Brief Part 1 says the in-house team is four developers, supported by an external agency. A good answer names one likely seam in the portal, says which groups sit either side of it, and says whether that seam is where the design needs it, and if not, who would need to talk more often.

Reference: Conway, M. E. (1968) 'How do committees invent?', Datamation, 14(4), pp. 28 to 31.

Norman: show what is possible, then confirm it

Why it matters: if tenants cannot see what to do, or cannot tell it worked, they phone instead.

AFFORDANCES AND SIGNIFIERS

Make the possible action obvious

An affordance is what something lets you do; a signifier tells you it is there.

A photo button that looks like a button gets pressed.

FEEDBACK

Tell people what just happened

Every action needs a visible, immediate result.

Try this: what feedback does a tenant get today after sending the web form? (Case Brief Part 1)

FIREBRAND LYCEUM EDUCATION GROUP

4

The idea. Norman separates what an object lets you do (an affordance) from what tells you that you can do it (a signifier), and stresses feedback: telling people straight away what has just happened. He added 'signifier' in the 2013 edition partly because 'affordance' had become loosely used in screen design, so use the two words carefully.

Why it matters. If tenants cannot see what to do on a screen, or cannot tell whether it worked, they give up and phone instead, which is the cost the portal is meant to cut.

Try this. Read how the web form works today in Case Brief Part 1, and what Maureen Hughes says tenants want in Part 2. A good answer says what feedback a tenant gets today after sending the form, then proposes feedback at two moments: straight after submitting, and later while the repair is in progress. Say how each would be shown on a phone screen.

Reference: Norman, D. A. (2013) The Design of Everyday Things. Revised and expanded edition. New York: Basic Books. First published in 1988 as The Psychology of Everyday Things.

You will use these ideas in Session 2, when your portal task flow becomes wireframes and a clickable prototype.

Evans: one language for experts and code

A GLOSSARY ENTRY: RIVERSIDE LEISURE CENTRE

(1) Term: class session

(2) Means: one run of a class at a set time in a set studio (Spin, Tuesday 18:00, Studio 2).

(3) Not the same as: class type, the kind of class (Spin).

(4) Agreed with: the bookings manager and the instructors.

1. The idea: experts and developers use the same words in talk, diagrams and code (Evans, 2003).
2. Define each term once, in the experts' words.
3. Why it matters: one word with two meanings becomes two bugs.
4. Try this: write the entry for 'Emergency'. Who must agree it?

FIREBRAND LYCEUM EDUCATION GROUP

5

The idea. Evans calls the shared vocabulary of domain experts and developers the ubiquitous language. The same terms are used in conversation, in diagrams and in the code, and a change in the language is treated as a change to the model. A glossary agreed with the experts is a simple way to start.

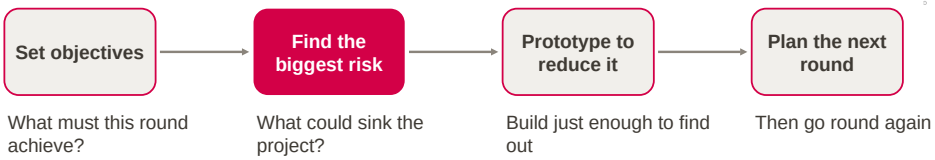
Why it matters. When one word means two things to two teams, the software ends up with two meanings as well, and the bugs that come with them. The example on the slide is from the leisure centre used in today's worked examples: 'class session' and 'class type' sound alike but are different things to the bookings team.

Try this. Write the glossary entry for 'Emergency' at Kestrel Moor Homes, using the Repair Priority Policy in Case Brief Part 4. A good entry has the four parts on the slide: the term, what it means in the experts' words, what it is not the same as, and who must agree it. Say why each person you name must agree it.

Reference: Evans, E. (2003) Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley.

Boehm's spiral: let risk choose the next step

Why it matters: design effort goes where failure would cost most (Boehm, 1988).



What must this round achieve?

What could sink the project?

Build just enough to find out

Then go round again

Try this: what is the biggest risk in the portal's first slice, and what is the cheapest prototype that would reduce it?

FIREBRAND LYCEUM EDUCATION GROUP

6

The idea. Boehm's spiral model is driven by risk. Each round sets objectives and constraints, finds and reduces the biggest risks (often with a prototype), develops and checks a little more, then plans the next round. The slide simplifies his four quadrants into four steps. Much of today's iterative practice follows the same logic.

Why it matters. Design effort is limited. The spiral tells you to spend it where failure would cost most, and to find out cheaply before you build properly.

Try this. Think about the first release of the Kestrel Moor portal, or of your own project. A good answer names one specific risk (something that could make the project fail, not a feature), says why it is the biggest, and describes the cheapest prototype or test that would reduce it, with what result would make you change course. Listing features instead of risks is the usual mistake.

Reference: Boehm, B. W. (1988) 'A spiral model of software development and enhancement', Computer, 21(5), pp. 61 to 72.

Brooks: no silver bullet

ESSENCE AND ACCIDENTS

> **The hard part of software is deciding what to build, not writing it in code.**

Brooks (1987) separates essential complexity, in the problem itself, from accidental complexity, from our tools. Better tools mostly remove the accidental kind.

Try this: which part of the portal stays hard whatever language or AI tool you use? Why?

Contested: some argue newer tools have shifted the balance.

FIREBRAND LYCEUM EDUCATION GROUP

7

The idea. Brooks separates essential complexity, which is in the problem itself, from accidental complexity, which comes from our tools and methods. His words: 'I believe the hard part of building software to be the specification, design, and testing of this conceptual construct, not the labor of representing it and testing the fidelity of the representation.' He argued that no single technology or method would give a tenfold improvement within a decade, because most of the difficulty is essential.

Why it matters. Better tools, including AI coding assistants, mostly remove accidental work. The analysis you did in Session 1, deciding what the system must do and what the rules really are, does not go away.

Contested. People still debate how far newer tools have reduced the accidental work; the claim that essential complexity remains is widely accepted.

Try this. A good answer names one part of the portal, or of your own project, that would stay hard whatever language or tool you used, and explains what makes it hard: rules, people or decisions rather than code.

Reference: Brooks, F. P. (1987) 'No Silver Bullet: Essence and Accidents of Software Engineering', Computer, 20(4), pp. 10 to 19.